FaceLit — diseño funcional y flujos actuales

Corte de revisión: 4 de noviembre de 2026. Se cotejó el diseño funcional con los árboles limpios de las ramas `main` disponibles localmente. Una pantalla existente no garantiza persistencia ni aceptación de todas las operaciones.

Repositorio	Rama	HEAD consultado
FaceLit	`main`	`6efd92cdd7e4e358b131cbe860cd8585d13e5550`
FaceLit-Backend	`main`	`2ba739853a808c8e5902f70eae98706a612f91e6`
micro-facial	`main`	`166e4a7f2245456277d57015259c678d8febfa9f`
FaceLit-DB	`main`	`cd069ae0cb4947d4b7ebbff17e9d6dc280ab86fd`

Navegación y perfiles

El cliente presenta funciones según la sesión y el rol. Aprendiz, instructor y coordinador comparten autenticación, perfil, idioma/tema y acceso a notificaciones, con menús distintos. La autorización efectiva se debe resolver también en Spring Security y en los servicios que comprueban el alcance del usuario sobre una ficha, sesión o recurso.

La interfaz se organiza por rutas Expo y áreas de funcionalidad; pantallas delegan coordinación a hooks y a servicios HTTP/estado compartido. Esta organización no debe describirse como cuatro aplicaciones diferentes.

Autenticación y alta de usuarios en `main`

El login se expone en `POST /api/auth/login`. La administración crea usuarios desde `POST /api/admin/users`, dentro del módulo administrativo; no corresponde a un registro público. La recuperación de acceso usa endpoints bajo `/api/auth` (`request-recovery`, `verify-token`, `reset-password`, entre otros). `POST /api/auth/privacy-acceptance-status` consulta el estado de aceptación de privacidad; por sí solo no registra una nueva aceptación.

En el `main` consultado no aparecen los controladores públicos `/register`, `/verify-email`, `/resend-code` ni las rutas `/api/consent/*` descritas en una revisión local anterior. Por tanto, este documento no presenta la verificación por correo ni el consentimiento de acudiente como flujos disponibles en la versión actual de `main`.

Preparación de información académica con CSV

La plantilla documentada usa la cabecera:

```
tipo,documento,nombre,apellido,correo,programa_codigo,ficha_codigo,instructor_tipo
```

Las filas admiten tipos documentados como programa, ficha, aprendiz e instructor. El flujo del backend conserva una fachada transaccional que coordina análisis sintáctico, validaciones, importación de estructura académica y personas, transferencias y notificaciones. La información se valida antes de persistir; errores se reportan por fila/categoría según el contrato actual.

Procedimiento operativo: descargar la plantilla desde la versión desplegada; completar códigos y tipos conforme a sus ejemplos; revisar datos y permisos antes de cargar; ejecutar en entorno de prueba; conservar el resultado de carga sin exponer correos o documentos de personas reales. El conteo de una carga no constituye por sí solo prueba de que todas las relaciones académicas sean correctas.

Registro facial del aprendiz

1. Inicia sesión con su cuenta y abre la función de registro facial.
2. Revisa la identidad y el aviso de tratamiento, y concede permiso a la cámara.
3. La interfaz prepara la captura y presenta retos.
4. El cliente envía imágenes Base64 y secuencias de imágenes en la solicitud al endpoint facial del backend.
5. El backend aplica autorización y coordina la llamada al servicio FastAPI.
6. El servicio evalúa la secuencia, genera la representación y persiste la referencia dentro del flujo autorizado.
7. El cliente muestra confirmación o el error devuelto.

El detalle de qué capturas temporales se retienen, por cuánto tiempo y cómo se eliminan debe contestarse con la política y el código vigente; no inferirlo de la existencia de un embedding.

Retos de movimiento vigentes

Código	Indicación
MOVE_UP	Mirar arriba y mantener la posición
MOVE_DOWN	Mirar abajo y mantener la posición
MOVE_CLOSER	Acercarse a la cámara
MOVE_AWAY	Alejarse de la cámara

La versión documentada admite una secuencia de **dos retos distintos**, con siete imágenes por reto. El usuario no realiza los cuatro movimientos en una sola secuencia. Los siete retos retirados son pestañear, abrir/cerrar la boca, asentir, negar, beso y movimientos laterales; el frontend los retiró del catálogo y el microservicio rechaza sus códigos, incluso si un cliente antiguo los envía.

La configuración observada usa umbral por defecto 0.65 para comparación de similitud. No equivale a 65 % de precisión. No hay aquí una evaluación biométrica representativa.

Sesión y asistencia

1. El instructor abre o selecciona una sesión permitida con ficha, ambiente y tiempos.
2. Se inicia la captura y se presenta al aprendiz la secuencia de retos.
3. El backend solicita la verificación facial al servicio especializado.
4. Con el resultado, el backend revisa rol, alcance, sesión, ventana temporal, entrada/salida previa y duplicados.
5. Si pasa las reglas, se persiste el evento y queda consultable para el alcance autorizado.
6. El usuario consulta la matriz/detalle y puede recargar la vista para comprobar persistencia.

La clasificación de puntualidad depende de hora de inicio y minutos permitidos para el registro configurados en la sesión. Una salida debe respetar las reglas de salida y la entrada previa. No manipular el reloj del servidor para demostrar una categoría.

Comportamiento síncrono y asíncrono

FaceLit combina mecanismos:

Parte	Comportamiento descrito
Frontend Axios/promesas/async-await	La solicitud de red es asíncrona para el cliente. La acción puede esperar la respuesta antes de avanzar; la pantalla puede seguir mostrando carga.
Caso de uso de asistencia	Secuencial respecto del resultado facial, reglas y persistencia: no confirmar antes del resultado requerido.
Backend Java → Python	Se usa Spring RestClient; la llamada espera la respuesta en el flujo de esa solicitud. No implica que todo el servidor atienda un usuario a la vez.
FastAPI → PostgreSQL	Rutas y SQLAlchemy/asyncpg usan interfaces asíncronas de entrada/salida.
Inferencia	Las llamadas del trabajador facial son síncronas en la aplicación y procesan las capturas en secuencia; async def no vuelve CPU-bound la inferencia no bloqueante automáticamente.
Correo	El listener citado usa evento transaccional posterior al commit y @Async; aplica a ese flujo, no necesariamente a todo correo.
WebSocket	Canal basado en eventos para actualizaciones seleccionadas; complementa HTTP y no sustituye el procesamiento de asistencia.

No afirmar “todo el sistema es asíncrono”, “async significa paralelo” o “todos los eventos corren en background”. La latencia puede incluir captura, red, decodificación, detección, inferencia, consultas y escritura; no se ha establecido una garantía general de tiempo máximo.

Base64 y cámara

El contrato actual transporta imágenes como texto Base64 dentro de solicitudes JSON. Base64 es la codificación de transporte adoptada por esta integración, no un requisito universal del reconocimiento facial. Cambiarla requiere coordinar cliente, backend y servicio facial.

La demostración debe usar HTTPS válido, permiso explícito de cámara, iluminación y encuadre adecuados. Si se usa una grabación, se debe identificar como evidencia previa, no presentarla como operación en vivo.

Evidencia y límites

Los contratos faciales se pueden cotejar con `FaceLit/features/facial/livenessChallenges.ts`, `FaceLit/shared/services/facialEmbeddingService.ts`, `FaceLit-Backend/src/main/java/com/FaceLit/backend/facial/` y `micro-facial/app/services/liveness_validator.py`. La administración de usuarios se puede cotejar con `FaceLit-Backend/src/main/java/com/FaceLit/backend/auth/controller/security/UserManagementController.java`; el login, con `auth/controller/roleandpermission/LoginController.java`. El equipo confirma que cámara, CSV, asistencia, acceso y correo funcionan en el sistema actual. Las pruebas sintéticas descritas aquí verifican reglas y transporte; no miden precisión biométrica poblacional.